

pandas: Data Manipulation

Objectives

- ✓ Data Preparation
- ✓ Concatenating
- ✓ Data Transformation
- ✓ Discretization and Binning
- ✓ String Manipulation
- ✓ Data Aggregation

About

- This lesson provides an in-depth exploration of how the **pandas** library supports data manipulation in data analysis.
- It explains the three main phases of data manipulation—**data preparation, data transformation, and data aggregation**—by presenting practical examples and demonstrating how to effectively use pandas functions to perform each stage.

Data Preparation

- Before performing data manipulation with the pandas library, data must first be prepared and organized into appropriate data structures. This ensures the data can be efficiently processed and analyzed using pandas tools.
- The different procedures for data preparation
 - Loading
 - Assembling
 - Merging
 - Concatenating
 - Combining
 - Reshaping (pivoting)
 - Removing

- The data contained in the pandas objects can be assembled in different ways:
 - *Merging*—The `pandas.merge()` function connects the rows in a dataframe based on one or more keys. This mode is very familiar to those who are confident with the SQL language, since it also implements join operations.
 - *Concatenating*—The `pandas.concat()` function concatenates the objects along an axis.
 - *Combining*—The `pandas.DataFrame.combine_first()` function allows you to connect overlapped data in order to fill in missing values in a data structure by taking data from another structure.

Merging

- Merging in **pandas** is equivalent to the SQL **JOIN** operation. It combines data from different tables (DataFrames) by linking rows using one or more shared keys.
- Similar to how SQL JOIN retrieves related data from relational database tables, pandas uses the `merge()` function to connect DataFrames based on common reference columns, producing a new combined tabular result.
- To perform merging, you first import the pandas library and create the DataFrames to be joined.

```
frame1 = pd.DataFrame( {'id':['ball','pencil','pen','mug','ashtray'],
                        'price': [12.33,11.44,33.21,13.23,33.62]})
```

frame1

	id	price	
0	ball	12.33	
1	pencil	11.44	
2	pen	33.21	
3	mug	13.23	
4	ashtray	33.62	

```
frame2 = pd.DataFrame( {'id':['pencil','pencil','ball','pen'],
                        'color': ['white','red','red','black']})
```

frame2

	id	color	
0	pencil	white	
1	pencil	red	
2	ball	red	
3	pen	black	

```
pd.merge(frame1,frame2)
```

	id	price	color
0	ball	12.33	red
1	pencil	11.44	white
2	pencil	11.44	red
3	pen	33.21	black

- The resulting DataFrame from a merge operation includes only the rows that share a common ID between both DataFrames. Along with the shared key column, all other columns from both DataFrames are included in the result. While **merge()** can automatically use common column names, it is usually necessary to explicitly specify the key column for merging by using the `on` parameter to define which column should be used as the basis for the join.

```
frame1 = pd.DataFrame( {'id': ['ball','pencil','pen','mug','ashtray'],  
                        'color': ['white','red','red','black','green'],  
                        'brand': ['OMG','ABC','ABC','POD','PPOD']})  
frame1
```

	id	color	brand
0	ball	white	OMG
1	pencil	red	ABC
2	pen	red	ABC
3	mug	black	POD
4	ashtray	green	PPOD

```
frame2 = pd.DataFrame( {'id':['pencil','pencil','ball','pen'],  
                        'brand': ['OMG','POD','ABC','POD']})  
frame2
```

	id	brand
0	pencil	OMG
1	pencil	POD
2	ball	ABC
3	pen	POD

```
#not get any results.  
pd.merge(frame1,frame2)
```

id	color	brand
----	-------	-------

- It is necessary to explicitly define the criteria for merging that pandas must follow, specifying the name of the key column in the on option.

```
pd.merge(frame1, frame2, on='id')
```

	id	color	brand_x	brand_y
0	ball	white	OMG	ABC
1	pencil	red	ABC	OMG
2	pencil	red	ABC	POD
3	pen	red	ABC	POD

```
pd.merge(frame1, frame2, on='brand')
```

	id_x	color	brand	id_y
0	ball	white	OMG	pencil
1	pencil	red	ABC	ball
2	pen	red	ABC	ball
3	,mug	black	POD	pencil
4	,mug	black	POD	pen

- The results of a merge operation can differ significantly depending on the merging criteria used. In some cases, the key columns in the two DataFrames do not share the same name.
- To handle this situation, pandas provides the **left_on** and **right_on** parameters in the merge() function, allowing you to explicitly specify which column in each DataFrame should be used as the merging key.

```
frame2.columns = ['sid', 'brand']
frame2
```

	sid	brand
0	pencil	OMG
1	pencil	POD
2	ball	ABC
3	pen	POD

```
pd.merge(frame1, frame2, left_on='id', right_on='sid')
```

	id	color	brand_x	sid	brand_y
0	ball	white	OMG	ball	ABC
1	pencil	red	ABC	pencil	OMG
2	pencil	red	ABC	pencil	POD
3	pen	red	ABC	pen	POD

By default, the merge() function performs an *inner join*; the keys in the result are the result of an intersection.

- Other possible options are the *left join*, the *right join*, and the *outer join*.
- The outer join produces the union of all keys, combining the effect of a left join with a right join. To select the type of join you have to use the **how** option.

```
frame2.columns = ['id', 'brand']
```

```
pd.merge(frame1, frame2, on='id')
```

	id	color	brand_x	brand_y
0	ball	white	OMG	ABC
1	pencil	red	ABC	OMG
2	pencil	red	ABC	POD
3	pen	red	ABC	POD

```
pd.merge(frame1,frame2,on='id',how='outer')
```

	id	color	brand_x	brand_y
0	,mug	black	POD	NaN
1	ashtray	green	PPOD	NaN
2	ball	white	OMG	ABC
3	pen	red	ABC	POD
4	pencil	red	ABC	OMG
5	pencil	red	ABC	POD

```
pd.merge(frame1,frame2,on='id',how='left')
```

	id	color	brand_x	brand_y
0	ball	white	OMG	ABC
1	pencil	red	ABC	OMG
2	pencil	red	ABC	POD
3	pen	red	ABC	POD
4	,mug	black	POD	NaN
5	ashtray	green	PPOD	NaN

```
pd.merge(frame1,frame2,on='id',how='right')
```

	id	color	brand_x	brand_y
0	pencil	red	ABC	OMG
1	pencil	red	ABC	POD
2	ball	white	OMG	ABC
3	pen	red	ABC	POD

- To merge multiple keys, you simply add a list to the on option.

```
pd.merge(frame1, frame2, on=['id', 'brand'], how='outer')
```



	id	color	brand
0	,mug	black	POD
1	ashtray	green	PPOD
2	ball	NaN	ABC
3	ball	white	OMG
4	pen	red	ABC
5	pen	NaN	POD
6	pencil	red	ABC
7	pencil	NaN	OMG
8	pencil	NaN	POD

• Merging on an Index

- In some situations, instead of using columns as merge keys, you can use the indexes of DataFrames as the basis for merging. To do this in pandas, you set the **left_index** and/or **right_index** parameters to True in the **merge()** function, depending on whether you want to use the index from one or both DataFrames as the key.

```
pd.merge(frame1, frame2, right_index=True, left_index=True)
```

	id_x	color	brand_x	id_y	brand_y
0	ball	white	OMG	pencil	OMG
1	pencil	red	ABC	pencil	POD
2	pen	red	ABC	ball	ABC
3	,mug	black	POD	pen	POD

Concatenating

- Another method of combining data is **concatenation**, which involves joining data structures along a particular axis. In NumPy, this operation is performed using the **concatenate() function**, which combines arrays into a single array.

```
array1 = np.arange(9).reshape((3,3))  
array1
```

```
array([[0, 1, 2],  
       [3, 4, 5],  
       [6, 7, 8]])
```

```
array2 = np.arange(9).reshape((3,3))+6  
array2
```

```
array([[ 6,  7,  8],  
       [ 9, 10, 11],  
       [12, 13, 14]])
```

```
np.concatenate([array1,array2],axis=1)
```

```
array([[ 0,  1,  2,  6,  7,  8],  
       [ 3,  4,  5,  9, 10, 11],  
       [ 6,  7,  8, 12, 13, 14]])
```

```
np.concatenate([array1,array2],axis=0)
```

```
array([[ 0,  1,  2],  
       [ 3,  4,  5],  
       [ 6,  7,  8],  
       [ 6,  7,  8],  
       [ 9, 10, 11],  
       [12, 13, 14]])
```


- In pandas, the presence of labeled axes in data structures like Series and DataFrames allows concatenation to be more flexible and powerful than with basic arrays.
- The **concat()** function is used to combine these labeled objects along a specified axis, enabling more generalized and structured data concatenation.

```
ser1 = pd.Series(np.random.rand(4), index=[1,2,3,4])  
ser1
```

```
0  
1 0.212987  
2 0.771457  
3 0.056762  
4 0.183005  
  
dtype: float64
```

```
ser2 = pd.Series(np.random.rand(4), index=[5,6,7,8])  
ser2
```

```
0  
5 0.029433  
6 0.969599  
7 0.160123  
8 0.472080  
  
dtype: float64
```

```
pd.concat([ser1,ser2])
```

```
0  
1 0.212987  
2 0.771457  
3 0.056762  
4 0.183005  
5 0.029433  
6 0.969599  
7 0.160123  
8 0.472080  
  
dtype: float64
```

- To create a hierarchical index on the axis of concatenation: use the **keys** option.

```
pd.concat([ser1,ser2],keys=[1,2])
```

0		
1	1	0.212987
	2	0.771457
	3	0.056762
	4	0.183005
2	5	0.029433
	6	0.969599
	7	0.160123
	8	0.472080

dtype: float64

- In the case of combinations between series along axis = 1, the keys become the column headers of the dataframe.

```
pd.concat([ser1,ser2],axis=1,keys=[1,2])
```

	1	2
1	0.212987	NaN
2	0.771457	NaN
3	0.056762	NaN
4	0.183005	NaN
5	NaN	0.029433
6	NaN	0.969599
7	NaN	0.160123
8	NaN	0.472080

- Another example

```
frame1 = pd.DataFrame(np.random.rand(9).reshape(3,3),
    index=[1,2,3],
    columns=['A', 'B', 'C'])
```

```
frame2 = pd.DataFrame(np.random.rand(9).reshape(3,3),
    index=[4,5,6],
    columns=['A', 'B', 'C'])
```

```
pd.concat([frame1, frame2])
```

	A	B	C
1	0.800706	0.715263	0.333170
2	0.170081	0.691602	0.761079
3	0.599590	0.359887	0.579775
4	0.467364	0.380181	0.054760
5	0.534632	0.442085	0.997265
6	0.833990	0.121499	0.019946

```
pd.concat([frame1, frame2], axis=1)
```

	A	B	C	A	B	C
1	0.800706	0.715263	0.333170	NaN	NaN	NaN
2	0.170081	0.691602	0.761079	NaN	NaN	NaN
3	0.599590	0.359887	0.579775	NaN	NaN	NaN
4	NaN	NaN	NaN	0.467364	0.380181	0.054760
5	NaN	NaN	NaN	0.534632	0.442085	0.997265
6	NaN	NaN	NaN	0.833990	0.121499	0.019946

Combining

- Some data combinations cannot be achieved through merging or concatenation, particularly when two datasets have fully or partially overlapping indexes.
- In such cases, pandas provides the **combine_first()** function (for Series), which aligns the data by index and fills missing values in one dataset with corresponding values from the other.

```
ser1 = pd.Series(np.random.rand(5), index=[1,2,3,4,5])  
ser1
```

```
0  
1  0.304632  
2  0.109872  
3  0.365013  
4  0.805351  
5  0.460251
```

dtype: float64

```
ser2 = pd.Series(np.random.rand(4), index=[2,4,5,6])  
ser2
```

```
0  
2  0.482589  
4  0.936603  
5  0.952857  
6  0.608593
```

dtype: float64

```
ser1.combine_first(ser2)
```

```
0  
1  0.304632  
2  0.109872  
3  0.365013  
4  0.805351  
5  0.460251  
6  0.608593
```

dtype: float64

```
ser2.combine_first(ser1)
```

```
0  
1  0.304632  
2  0.482589  
3  0.365013  
4  0.936603  
5  0.952857  
6  0.608593
```

dtype: float64

Pivoting

- Besides assembling data from multiple sources, another common operation in data preparation is **pivoting**.
- The original arrangement of values by rows or columns may not always suit analytical needs.
- Pivoting allows you to reorganize the dataset by transforming column values into rows or vice versa, enabling a more meaningful structure for analysis.

- **Pivoting with Hierarchical Indexing**
- Two basic operations:
 - Stacking—Rotates or pivots the data structure, converting columns to rows
 - Unstacking—Converts rows into columns

```
frame1 = pd.DataFrame(np.arange(9).reshape(3,3),
                      index=['white','black','red'],
                      columns=['ball','pen','pencil'])
frame1
```

	ball	pen	pencil
white	0	1	2
black	3	4	5
red	6	7	8

```
#Using the stack() function on the dataframe
ser1 = frame1.stack()
ser1
```

```

0
white  ball  0
      pen   1
      pencil 2
black  ball  3
      pen   4
      pencil 5
red    ball  6
      pen   7
      pencil 8

dtype: int64
```

```
ser1.unstack()
```

	ball	pen	pencil
white	0	1	2
black	3	4	5
red	6	7	8

```
#unstack on a different level
ser1.unstack(0)
```

	white	black	red
ball	0	3	6
pen	1	4	7
pencil	2	5	8

Pivoting from “Long” to “Wide” Format

- Many datasets are commonly stored in formats such as CSV files or database tables, where data is recorded line by line over time—such as instrument readings, iterative calculation results, manual entries, or log files.
- These datasets often contain repeated values across columns in successive rows.
- This tabular structure is known as the **long (or stacked) format**, where each row represents a single observation and data may be distributed across multiple rows for the same entity.

```
longframe = pd.DataFrame({ 'color':['white','white','white',
'red','red','red',
'black','black','black'],
'item':['ball','pen','mug',
'ball','pen','mug',
'ball','pen','mug'],
'value': np.random.rand(9)})
```

longframe

	color	item	value
0	white	ball	0.532077
1	white	pen	0.754936
2	white	mug	0.554992
3	red	ball	0.599807
4	red	pen	0.241771
5	red	mug	0.490109
6	black	ball	0.317861
7	black	pen	0.821009
8	black	mug	0.771687

Pandas provides the pivot() function to transform a DataFrame from **long (stacked) format** to **wide format**.

This function reshapes the data by using one or more specified columns as keys, reorganizing values into a new structure where unique column values become new columns in the resulting DataFrame.

```
wideframe = longframe.pivot(index='color',columns='item')
wideframe
```

	value		
item	ball	mug	pen
color			
black	0.317861	0.771687	0.821009
red	0.599807	0.490109	0.241771
white	0.532077	0.554992	0.754936

Removing

- The last stage of data preparation is the removal of columns and rows.
- To remove a **column**: apply the **del** command to the dataframe with the column name specified.
- To remove an unwanted **row**: use the **drop() function** with the label of the corresponding index as an argument.

```
frame1 = pd.DataFrame(np.arange(9).reshape(3,3),  
                      index=['white','black','red'],  
                      columns=['ball','pen','pencil'])  
frame1
```

	ball	pen	pencil
white	0	1	2
black	3	4	5
red	6	7	8



```
del frame1['ball']  
frame1
```

	pen	pencil
white	1	2
black	4	5
red	7	8

```
frame1.drop('white')
```

	pen	pencil
black	4	5
red	7	8

Data Transformation

- After preparing and reorganizing data, the next stage of data manipulation is **data transformation**.
- This phase focuses on modifying the values within the data structure to make them suitable for analysis.
- Common tasks include detecting and handling duplicate or invalid values (by removing or replacing them), modifying indexes, and processing numerical and string data. Pandas provides various functions to address these common transformation challenges efficiently.

Removing Duplicates

- Duplicate rows can appear in a DataFrame for various reasons, especially in large datasets where detecting them manually is difficult.
- Pandas provides dedicated tools to identify and manage duplicate data efficiently, making it easier to analyze and clean large data structures.

```
dframe = pd.DataFrame({'color': ['white', 'white', 'red', 'red', 'white'],  
                        'value': [2, 1, 3, 3, 2]})  
dframe
```

	color	value
0	white	2
1	white	1
2	red	3
3	red	3
4	white	2

- The duplicated() function in pandas identifies duplicate rows in a DataFrame.
- It returns a Boolean Series where each value corresponds to a row: True indicates that the row is a duplicate (excluding its first occurrence), while False indicates that the row is unique up to that point.
- Since duplicated() returns a **Boolean Series**, it is especially useful for filtering operations. By using this Boolean result, you can easily select and display only the rows that are duplicated within the DataFrame.

```
dframe.duplicated()
```

0

0 False

1 False

2 False

3 True

4 True

dtype: bool

```
dframe[dframe.duplicated()]
```

	color	value
3	red	3
4	white	2

Mapping

- Pandas provides several functions that use **mapping** to perform data transformations. Mapping involves creating relationships between different values, allowing one value to be associated with a specific label or string.
- The most suitable structure for defining these value correspondences is a Python **dict object**, which efficiently stores key-value pairs for mapping operations.
- Syntax:

```
map = {  
    'label1' : 'value1',  
    'label2' : 'value2',  
    ...  
}
```

• Replacing Values via Mapping

- In many datasets, some values may not meet analysis requirements—for example, text in a foreign language, synonymous labels, or values formatted incorrectly.
- In such situations, performing a **replace** operation is often necessary to standardize, clean, or convert values into a consistent and desired format for analysis.

```
frame = pd.DataFrame({'item': ['ball', 'mug', 'pen', 'pencil', 'ashtray'],  
                      'color': ['white', 'rosso', 'verde', 'black', 'yellow'],  
                      'price': [5.56, 4.20, 1.30, 0.56, 2.75]})  
frame
```

	item	color	price
0	ball	white	5.56
1	mug	rosso	4.20
2	pen	verde	1.30
3	pencil	black	0.56
4	ashtray	yellow	2.75

- To replace the incorrect values with new values, it is necessary to define a mapping of correspondences, containing as a key the new values.
- Use the `replace()` function with the mapping as an argument

	item	color	price
0	ball	white	5.56
1	mug	rosso	4.20
2	pen	verde	1.30
3	pencil	black	0.56
4	ashtray	yellow	2.75

```
newcolors = {  
    'rosso': 'red',  
    'verde': 'green'  
}
```

```
frame.replace(newcolors)
```

	item	color	price
0	ball	white	5.56
1	mug	red	4.20
2	pen	green	1.30
3	pencil	black	0.56
4	ashtray	yellow	2.75

- the replacement of NaN values with another value, for example 0.

```
ser = pd.Series([1,3,np.nan,4,6,np.nan,3])  
ser
```

	0
0	1.0
1	3.0
2	NaN
3	4.0
4	6.0
5	NaN
6	3.0

dtype: float64

```
ser.replace(np.nan,0)
```

	0
0	1.0
1	3.0
2	0.0
3	4.0
4	6.0
5	0.0
6	3.0

dtype: float64

• Adding Values via Mapping

- In this scenario, mapping is used to **create new values** in one column based on the values of another column.
- A mapping structure (typically defined separately, such as with a dictionary) is applied to establish the relationship between existing values and the new corresponding values to be added.

- Define then a dict object that contains a list of prices for each type of item

```
frame = pd.DataFrame({ 'item':['ball','mug','pen','pencil','ashtray'],  
                      'color':['white','red','green','black','yellow']})  
frame
```

	item	color
0	ball	white
1	mug	red
2	pen	green
3	pencil	black
4	ashtray	yellow

```
prices = {  
    'ball' : 5.56,  
    'mug' : 4.20,  
    'bottle' : 1.30,  
    'scissors' : 3.41,  
    'pen' : 1.30,  
    'pencil' : 0.56,  
    'ashtray' : 2.75  
}
```

```
frame['price'] = frame['item'].map(prices)  
frame
```

	item	color	price
0	ball	white	5.56
1	mug	red	4.20
2	pen	green	1.30
3	pencil	black	0.56
4	ashtray	yellow	2.75

• Rename the Indexes of the Axes

- Just like data values in a Series or DataFrame, axis labels (such as index or column names) can also be transformed using mapping.
- Pandas provides the **rename()** function for this purpose, which accepts a dictionary (dict) as a mapping argument to replace existing index or column labels with new ones.

frame			
	item	color	price
0	ball	white	5.56
1	mug	red	4.20
2	pen	green	1.30
3	pencil	black	0.56
4	ashtray	yellow	2.75

```
reindex = {  
    0: 'first',  
    1: 'second',  
    2: 'third',  
    3: 'fourth',  
    4: 'fifth'}
```

frame.rename(reindex)			
	item	color	price
first	ball	white	5.56
second	mug	red	4.20
third	pen	green	1.30
fourth	pencil	black	0.56
fifth	ashtray	yellow	2.75

```
recolumn = {  
    'item': 'object',  
    'price': 'value'}
```

frame.rename(index=reindex, columns=recolumn)			
	object	color	value
first	ball	white	5.56
second	mug	red	4.20
third	pen	green	1.30
fourth	pencil	black	0.56
fifth	ashtray	yellow	2.75

Discretization and Binning

- A more advanced transformation technique is **discretization**, which converts continuous numerical data into discrete categories.
- This is particularly useful when dealing with large volumes of sequential or highly precise data.
- By dividing the range of values into **smaller intervals (bins)**, you can group observations into categories and then analyze their frequencies or statistical properties.
- Discretization simplifies data analysis by making large or continuous datasets more manageable and interpretable.

- Example:

- Result: reading of an experimental value between 0 and 100

```
results = [12,34,67,55,28,90,99,12,3,56,74,44,87,23,49,89,87]
```

- If experimental values range from 0 to 100, the interval can be divided uniformly into four equal bins to facilitate analysis. These bins would cover the ranges 0–25, 26–50, 51–75, and 76–100. This division allows the data to be grouped into discrete categories for easier comparison and statistical evaluation.

```
bins = [0,25,50,75,100]
```

- use a special function called `cut()` and apply it to the array of results, also passing the bins.

```
cat = pd.cut(results, bins)
cat
```

```
[(0, 25], (25, 50], (50, 75], (50, 75], (25, 50], ..., (75, 100], (0, 25], (25, 50], (75, 100], (75, 100]]
Length: 17
Categories (4, interval[int64, right]): [(0, 25] < (25, 50] < (50, 75] < (75, 100]]
```

- The `cut()` function in pandas returns a **Categorical** object. This object can be viewed as an array of labels representing the bins assigned to each value. Internally, it contains two main components:
 - A **categories** array, which stores the bin labels.
 - A **codes** array, which stores numerical codes corresponding to the bin assigned to each original value.
- Each code indicates which bin a particular element belongs to, providing an efficient internal representation of the categorized data.

```
cat.categories
```

```
IntervalIndex([(0, 25], (25, 50], (50, 75], (75, 100]], dtype='interval[int64, right]')
```

```
cat.codes
```

```
array([0, 1, 2, 2, 1, 3, 3, 0, 0, 2, 2, 1, 3, 0, 1, 3, 3], dtype=int8)
```

- To know the occurrences for each bin, that is, how many results fall into each category, you have to use the **value_counts()** function.

```
pd.value_counts(cat)
```

```
/tmp/ipython-input-406/764800318.py:1: FutureWarning:  
pd.value_counts(cat)
```

	count
(75, 100]	5
(0, 25]	4
(25, 50]	4
(50, 75]	4

```
dtype: int64
```


Detecting and Filtering Outliers

- During data analysis, the need to detect the presence of abnormal values in a data structure often arises.
- By way of example, create a dataframe with three columns of 1,000 completely random values

```
randframe = pd.DataFrame(np.random.randn(1000,3))
```

- **describe()** function: the statistics for each column.

```
randframe.describe()
```

	0	1	2
count	1000.000000	1000.000000	1000.000000
mean	-0.005656	0.008757	-0.003869
std	1.026473	1.031696	1.006677
min	-3.118488	-3.254731	-3.242015
25%	-0.710046	-0.651451	-0.684653
50%	0.025901	0.002907	-0.036309
75%	0.753492	0.709177	0.665548
max	3.762730	3.558223	3.301128

- use the **std()** function.

```
randframe.std()
```

	0
0	1.026473
1	1.031696
2	1.006677

dtype: float64

- Now you apply the filtering of all the values of the dataframe, applying the corresponding standard deviation for each column.
 - any() function

```
randframe[(np.abs(randframe) > (3*randframe.std())).any(axis=1)]
```

	0	1	2
24	3.762730	0.786523	1.421078
490	-0.641704	-3.254731	-1.056590
520	0.451220	1.588251	3.301128
534	3.090554	1.065618	0.606971
641	-3.106227	0.743313	0.835257
763	1.075747	-0.187635	-3.242015
867	0.529419	3.558223	1.763683
878	-3.118488	0.431154	-0.003772

Permutation

- The operations of permutation (random reordering) on a series or on the rows of a dataframe are easy to do using the **numpy.random.permutation()** function.

```
#create a dataframe containing integers in ascending order.  
nframe = pd.DataFrame(np.arange(25).reshape(5,5))  
nframe
```

	0	1	2	3	4
0	0	1	2	3	4
1	5	6	7	8	9
2	10	11	12	13	14
3	15	16	17	18	19
4	20	21	22	23	24



```
#create an array of five integers from 0 to 4,  
#arranged in random order with the permutation() function.  
new_order = np.random.permutation(5)
```

```
new_order
```

```
array([2, 1, 3, 0, 4])
```

- Apply it to the dataframe on all lines, using the **take()** function.

```
nframe.take(new_order)
```

	0	1	2	3	4
2	10	11	12	13	14
1	5	6	7	8	9
3	15	16	17	18	19
0	0	1	2	3	4
4	20	21	22	23	24

- Submit even a portion of the entire dataframe to a permutation. It generates an array that has a sequence limited to a certain range, for example, in this case from 2 to 4.

```
new_order = [3,4,2]
```

```
nframe.take(new_order)
```

	0	1	2	3	4
3	15	16	17	18	19
4	20	21	22	23	24
2	10	11	12	13	14

Random Sampling

- After seeing how to extract portions of a DataFrame through permutation, another common need—especially with very large datasets—is **random sampling**. A quick way to generate random indices for sampling is by using NumPy's **np.random.randint()** function, which produces random integers that can be used to select rows from the DataFrame.

```
sample = np.random.randint(0, len(nframe), size=3)
```

```
sample
```

```
array([3, 3, 2])
```

```
nframe.take(sample)
```

	0	1	2	3	4
3	15	16	17	18	19
3	15	16	17	18	19
2	10	11	12	13	14

String Manipulation

- Python is widely appreciated for its simplicity and effectiveness in handling strings and text processing.
- Many common text operations can be performed using built-in functions.
- For more advanced pattern matching and text manipulation tasks, however, regular expressions are required.

Built-in Methods for String Manipulation

- When working with composite strings, it is often necessary to separate their components and assign them to appropriate variables.
- The `split()` function enables this by dividing a string into parts based on a specified separator, such as a comma, making it easier to extract and organize individual elements.

```
text = '16 Bolton Avenue , Boston'
```

```
text.split(',')
```

```
['16 Bolton Avenue ', ' Boston']
```

- Sometimes, after splitting a string, extra whitespace (such as spaces or newline characters) may remain in the resulting elements.
- To resolve this issue, the **split() function** can be used together with the **strip() function**, which removes leading and trailing whitespace, ensuring cleaner and properly formatted text values.

```
tokens = [s.strip() for s in text.split(',')]
```

tokens

```
['16 Bolton Avenue', 'Boston']
```

```
address, city = [s.strip() for s in text.split(',')]
```

address

```
'16 Bolton Avenue'
```

city

```
'Boston'
```

```
address + ',' + city
```

```
'16 Bolton Avenue,Boston'
```


- While simple concatenation works well for combining two or three strings, it becomes less practical when dealing with many parts.
- The **join()** function is more efficient. It is applied to a chosen separator string, which is used to connect multiple strings into a single combined result.

```
strings = ['A+', 'A', 'A-', 'B', 'BB', 'BBB', 'C+']
```

```
';'.join(strings)
```

```
'A+;A;A-;B;BB;BBB;C+'
```

- Another common string operation is searching for substrings within text. Python provides the `in` keyword as a simple and effective way to check whether a specific substring exists within a string.

```
'Boston' in text
```

```
True
```

- detecting substrings: index() and find()

```
['16 Bolton Avenue', 'Boston']
```

```
text.index('Boston')  
19
```

```
text.find('Boston')  
19
```

```
text.index('New York') #ValueError: substring not found
```

```
-----  
ValueError                                Traceback (most recent call last)  
/tmp/ipython-input-406/731579938.py in <cell line: 0>()  
----> 1 text.index('New York')
```

```
ValueError: substring not found
```

Next steps: [Explain error](#)

```
text.find('New York') #-1 not found
```

```
-1
```

- The **count()** function

'16 Bolton Avenue , Boston'

```
text.count('e')
```

2

```
text.count('Avenue')
```

1

- The **replace()** function

```
text.replace('Avenue', 'Street')
```

'16 Bolton Street , Boston'

```
text.replace('1', '')
```

'6 Bolton Avenue , Boston'

Regular Expressions

- Regular expressions (regex) offer a powerful and flexible way to search for and match string patterns within text.
- A regex is a specially formatted string written according to the rules of the regular expression language.
- In Python, regex operations are handled by the built-in re module. To use regular expressions, you must first import this module before performing pattern-matching operations:

```
import re
```

- There are three categories in re
 - Pattern matching
 - Substitution
 - Splitting

- There is a **split()** function even for the re module that performs the same operations, but it can accept a regex pattern as the criteria of separation, which makes it considerably more flexible.

```
text = "This is an\t odd \n text!"
```

```
re.split(r'\s+', text)
```

```
['This', 'is', 'an', 'odd', 'text!']
```

```
#compile the regex function with the re.compile() function  
regex = re.compile(r'\s+')
```

```
regex.split(text)
```

```
['This', 'is', 'an', 'odd', 'text!']
```

- if you want to find in a string all the words starting with “A” uppercase, or for example, with “a” regardless of whether it's upper- or lowercase

```
text = 'This is my address: 16 Bolton Avenue, Boston'
```

```
re.findall(r'A\w+',text)
```

```
['Avenue']
```

```
re.findall(r'[A,a]\w+',text)
```

```
['address', 'Avenue']
```

- There are two other functions related to the `findall()` function—**`match()`** and **`search()`**.
- While `findall()` returns all matches within a list, the **`search()` function** returns only the first match.

```
text = 'This is my address: 16 Bolton Avenue, Boston'
```

```
re.search(r'[A,a]\w+',text)
```

```
<re.Match object; span=(11, 18), match='address'>
```

```
search = re.search(r'[A,a]\w+',text)
```

```
search.start()
```

```
11
```

```
search.end()
```

```
18
```

```
text[search.start():search.end()]
```

```
'address'
```

- The **match() function** performs matching only at the beginning of the string; if there is no match to the first character, it goes no farther in research within the string. If you do not find a match, then it will not return any objects.

```
re.match(r'[A,a]\w+',text)
```

```
re.match(r'T\w+',text)
```

```
<re.Match object; span=(0, 4), match='This'>
```

```
match = re.match(r'T\w+',text)
```

```
text[match.start():match.end()]
```

```
'This'
```


Data Aggregation

- The final stage of data manipulation is **data aggregation**, which transforms a collection of values into a single result, such as with functions like `sum()`, `mean()`, or `count()`. While these basic functions perform simple aggregations, a more structured and powerful approach involves **categorizing data into groups** before applying aggregation functions.
- In pandas, this process is handled by the highly flexible and efficient **GroupBy** tool. Although it is conceptually similar to SQL's **GROUP BY** clause in relational databases, pandas offers far greater flexibility, allowing more complex and advanced operations on grouped data through Python's programming capabilities.

GroupBy

- This section explains the **GroupBy** process in detail, describing its internal mechanism as the **split-apply-combine** procedure. This approach can be understood as three main phases:
 - **Split** – Divide the dataset into groups based on specified criteria.
 - **Apply** – Perform a function or transformation on each group independently.
 - **Combine** – Merge the results from each group into a single aggregated output.
- This structured workflow makes GroupBy a powerful and systematic tool for grouped data analysis.

- Example

```
frame = pd.DataFrame({ 'color': ['white','red','green','red','green'],  
  'object': ['pen','pencil','pencil','ashtray','pen'],  
  'price1' : [5.56,4.20,1.30,0.56,2.75],  
  'price2' : [4.75,4.12,1.60,0.75,3.15]})
```

frame

	color	object	price1	price2
0	white	pen	5.56	4.75
1	red	pencil	4.20	4.12
2	green	pencil	1.30	1.60
3	red	ashtray	0.56	0.75
4	green	pen	2.75	3.15

- To calculate the average of the price1 column using group labels listed in the color column.

```
group = frame['price1'].groupby(frame['color'])
```

```
group
```

```
<pandas.core.groupby.generic.SeriesGroupBy object at 0x786cd528e300>
```

- When you perform a grouping operation, the result is a **GroupBy** object. At this stage, no actual calculation has yet been executed; instead, pandas has simply organized the data into groups based on the specified key (for example, grouping all rows with the same value of *color* together).
- To inspect how the DataFrame has been divided, you can access the `groups` attribute of the GroupBy object. This attribute reveals the internal structure of the grouping, showing which rows belong to each group.

```
group.groups
```

```
{'green': [2, 4], 'red': [1, 3], 'white': [0]}
```

- Examination with mean() and sum()

```
group.mean()
```

	price1
color	
green	2.025
red	2.380
white	5.560

dtype: float64

```
group.sum()
```

	price1
color	
green	4.05
red	4.76
white	5.56

dtype: float64

Hierarchical Grouping

- Have seen how to group the data according to the values of a column as a key choice.
- The same thing can be extended to multiple columns, that is, make a grouping of multiple keys hierarchical.

```
ggroup = frame['price1'].groupby([frame['color'], frame['object']])
```

```
ggroup.groups
```

```
{('green', 'pen'): [4], ('green', 'pencil'): [2], ('red', 'ashtray'): [3], ('red', 'pencil'): [1], ('white', 'pen'): [0]}
```

```
ggroup.sum()
```

		price1
color	object	
green	pen	2.75
	pencil	1.30
red	ashtray	0.56
	pencil	4.20
white	pen	5.56

```
dtype: float64
```

- Although grouping has been applied to a single column so far, the **GroupBy** operation can also be performed on multiple columns or even the entire DataFrame.
- If the grouped object does not need to be reused, it is often more practical to chain the grouping and aggregation steps together in a single statement, avoiding the creation of an intermediate GroupBy variable.

```
frame[['price1', 'price2']].groupby(frame['color']).mean()
```

	price1	price2
color		
green	2.025	2.375
red	2.380	2.435
white	5.560	4.750

```
frame.groupby(frame['color']).mean(numeric_only=True)
```

	price1	price2
color		
green	2.025	2.375
red	2.380	2.435
white	5.560	4.750

Group Iteration

- The GroupBy object supports an iteration to generate a sequence of two tuples containing the name of the group together with the data portion.

	color	object	price1	price2
0	white	pen	5.56	4.75
1	red	pencil	4.20	4.12
2	green	pencil	1.30	1.60
3	red	ashtray	0.56	0.75
4	green	pen	2.75	3.15

```
for name, group in frame.groupby('color'):
    print(name)
    print(group)
```

```
green
  color object  price1  price2
2 green  pencil    1.30    1.60
4 green   pen     2.75    3.15
red
  color object  price1  price2
1 red   pencil    4.20    4.12
3 red  ashtray    0.56    0.75
white
  color object  price1  price2
0 white   pen     5.56    4.75
```


Chain of Transformations

- At any stage of the GroupBy process, it is possible to select a single column for further operations.
- Whether the selection occurs before grouping, during grouping, or after aggregation, pandas allows flexible column selection throughout the workflow.
- This demonstrates the high flexibility and versatility of the grouping system provided by pandas.

```
frame['price1'].groupby(frame['color']).mean()
```

price1	
color	
green	2.025
red	2.380
white	5.560

dtype: float64

```
frame.groupby(frame['color'])['price1'].mean()
```

price1	
color	
green	2.025
red	2.380
white	5.560

dtype: float64

Functions on Groups

- Although many methods were not designed exclusively for use with **GroupBy**, they still work correctly when applied to the resulting Series objects.
- By selecting a specific column from a GroupBy object, you can obtain a Series and then apply standard Series methods to perform calculations. For example, you can compute group-wise quantiles using the **quantile()** function.

```
group = frame.groupby('color')
```

```
group['price1'].quantile(0.6)
```

	price1
color	
green	2.170
red	2.744
white	5.560

dtype: float64

- You can also define your own aggregation functions. Define the function separately and then pass it as an argument to the **range()** function.
- For example, calculate the range of the values of each group.

```
def range(series):  
    return series.max() - series.min()
```

```
group['price1'].agg(range)
```

price1	
color	
green	1.45
red	3.64
white	0.00

dtype: float64

